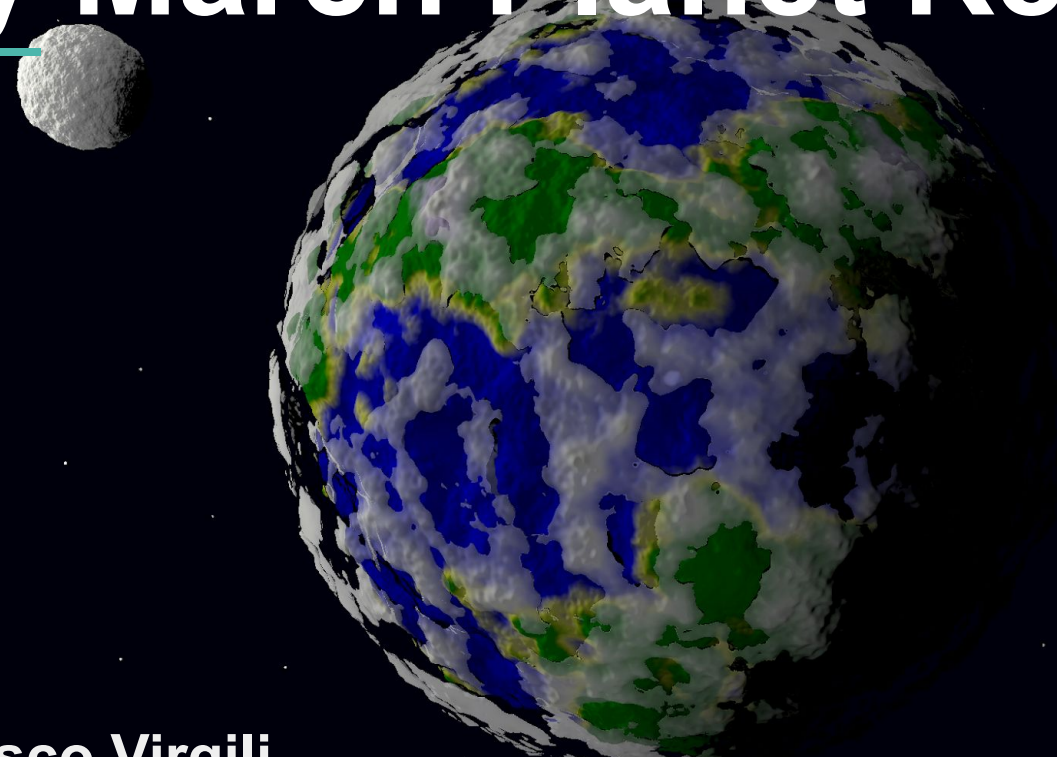


Ray March Planet Rendering

A 3D rendering of a planet with a blue, green, and white surface, set against a black background with a small grey sphere in the upper left.

Francesco Virgili

Matricola: 914367

Setup Fragment Shader

La scena è renderizzata esclusivamente tramite **fragment shader** applicato a un triangolo full-screen.

Normalizzazione coordinate:

```
uv = 2.0 * fragCoord / iResolution.xy - 1.0;
```

Correzione Aspect Ratio:

```
uv.x *= iResolution.x / iResolution.y;
```

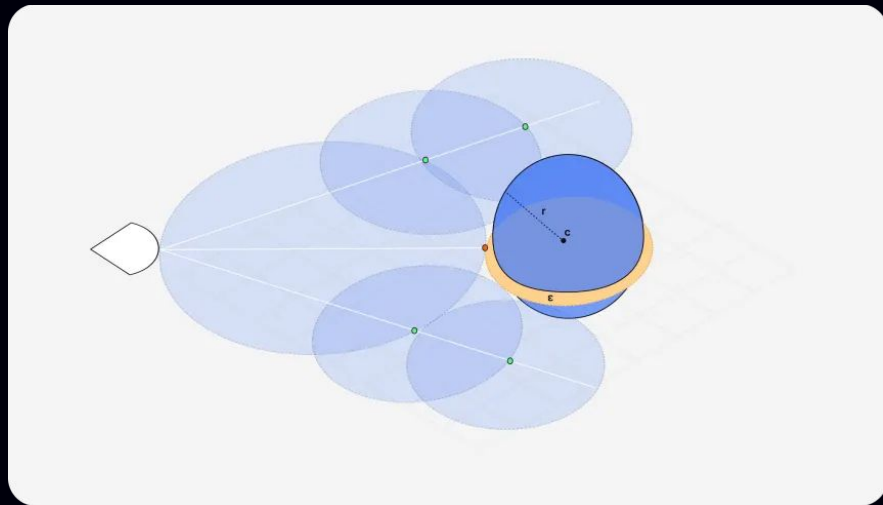
Come tecnica di rendering principale è stato utilizzato il **Ray Marching**.



Ray Marching

Il **Ray Marching** avanza lungo il raggio a passi variabili.

- Utilizza le **Signed Distance Functions (SDF)** per determinare la distanza dall'oggetto più vicino.
- Il raggio "marcia" in avanti di una distanza pari al valore restituito dalla SDF.
- Questo processo garantisce di non saltare mai oltre una superficie.



Sphere Tracing Visualization

Visualizzazione del raggio che si avvicina alla superficie della sfera.

Telecamera Orbitale

Funzione: pixelUvCoordinate.xy $\rightarrow$ cameraFunction $\rightarrow$ cameraOrigin.xyz, CameraDirection.xyz

hd (angolo orizzontale)

elv (angolo verticale)

Origine del raggio (Coordinate Sferiche to Cartesiane)

```
x = sin(hd) * cos(elv)
```

```
y = sin(elv)
```

```
z = cos(hd) * cos(elv)
```

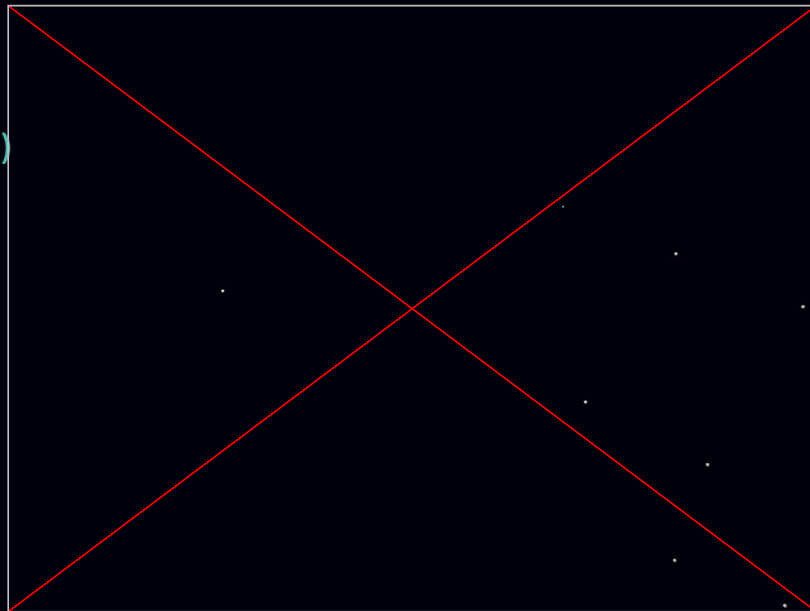
Vettore direzionale

```
fowardV = (centro - ro) / ||centro - ro||
```

```
UpV = (0, 1, 0)
```

```
rightV = UpV  $\times$  fowardV (prodotto vettoriale)
```

```
rd = norm(uv.x * rightV + uv.y * upV + focal * fowardV);
```



Ray March Main Loop

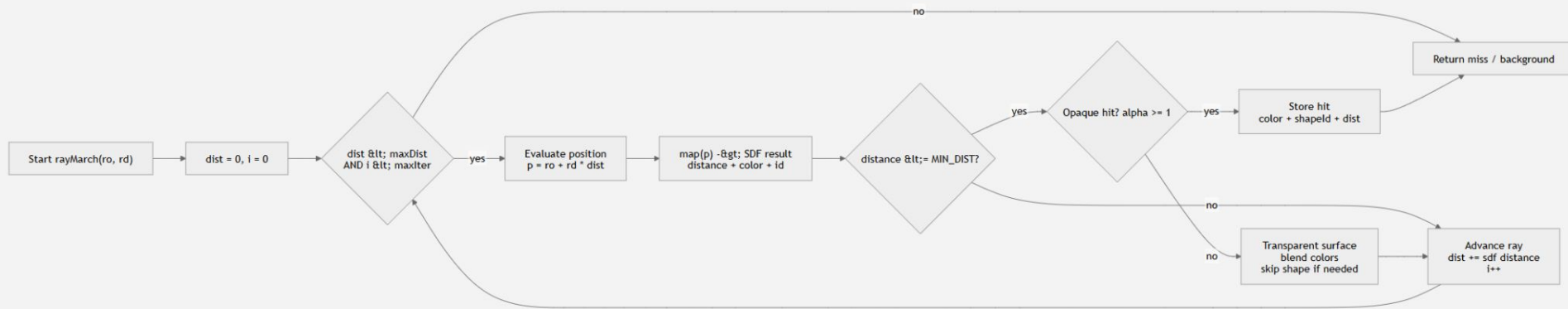


Diagramma del Flusso Principale
Implementazione del ciclo di rendering tramite SDF.

SDF Planet

Coordinate Transformation

$$p' = p - c$$

$$\begin{pmatrix} u' \\ v' \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix}$$

Rotazione divisa i tre rotazioni 2D per p.yz, p.xz, p.xy.

Distance Function (SDF)

$$d(p) = |p| - r * (1 + \text{noise})$$

Noise layering di Simplex Noise:

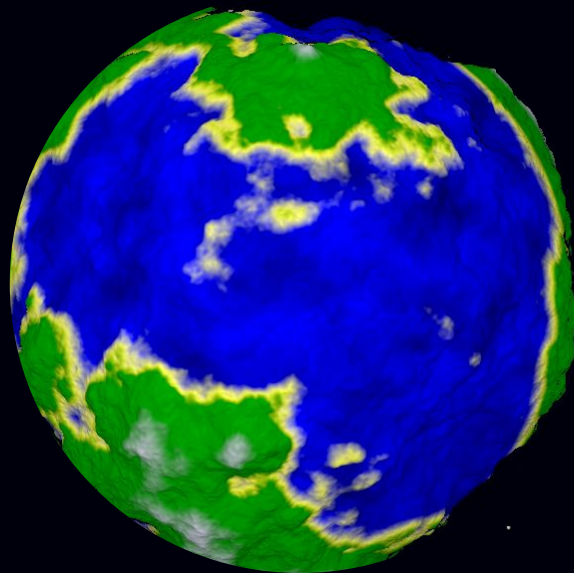
$$\text{noise}^2 + \text{FractalSimplexNoise}$$

Fractal Noise Parameters

- **a**: ampiezza (amplitude)
- **s**: frequenza (frequency)
- **r**: rotazione (rotation)

$$F(m) = \sum_{i=0}^3 a_i f(s_i R_i m)$$

Colors are mapped based on the noise values.



SDF Stelle

Modello Geometrico

Le stelle sono rappresentate come semplici sfere definite dalla funzione:

$$d(p) = |p| - r$$

Ripetizione Infinita

Vengono ripetute nello spazio mappando le coordinate del punto p tramite l'operatore modulo:

$$p' = \text{mod}(p, 2) - 1$$

Questo approccio permette di calcolare l'SDF una sola volta per mostrare un'intera volta celeste di stelle ripetute efficientemente.



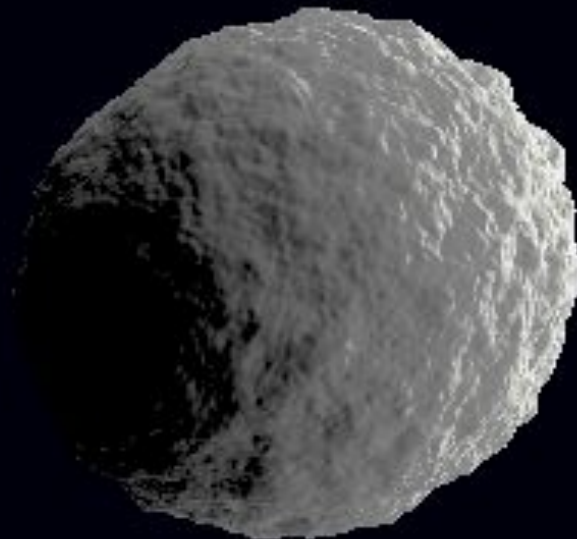
SDF Luna

Descrizione del Modello

Il modello della Luna utilizza un approccio simile a quello di **SDF Planet**, basato sul layering di noise per definire la superficie.

A differenza del pianeta, i colori rimangono costanti:

- Utilizzo di `Fractal Simplex Noise` per la rugosità superficiale.
- Mappatura cromatica fissa (scala di grigi) per simulare la regolite lunare.



Nuvole

Effetto Shell

Sfera con simplex noise ma per ottenere un effetto shell sottraiamo una sfera con lo stesso raggio ma noise invertite (scavato invece che in rilievo).

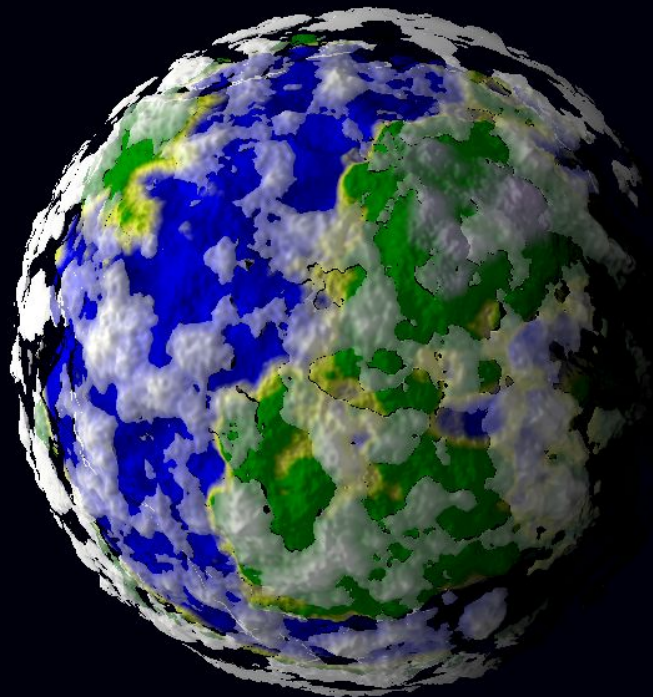
Unione delle noise

```
max(base, -oppbase)
```

Trasparenza Dinamica

La trasparenza viene variata in base al valore delle noise:

```
color.h = noise * 2.5 + 0.5
```



Illuminazione

Setup

Prendiamo la posizione del punto colpito:

```
pos = ro + rd * p.dist
```

Modello Phong Lighting

```
lighting = ambient * n + diffuse * m + specular * h
```

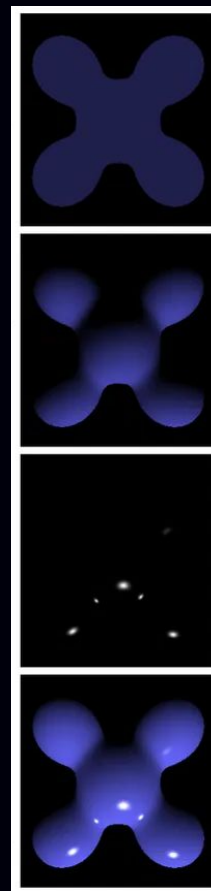
- Ambient: fisso
- Diffuse: $(\partial f / \partial x, \partial f / \partial y, \partial f / \partial z) \cdot (lp - pos / ||lp - pos||)$

Blinn-Phong Specular

$$I_s = K_s \cdot \max\left(0, \mathbf{N} \cdot \frac{\mathbf{L} + \mathbf{V}}{\|\mathbf{L} + \mathbf{V}\|}\right)^\alpha$$

Applicazione luce al colore

```
baseColor.rgb * lighting
```



Soft shadows

Algoritmo di Ombreggiatura

L'ombra viene calcolata campionando la distanza minima lungo il raggio verso la sorgente luminosa:

```
shadow = min(shadow, h.dist / t);
```

Avanzamento del raggio:

```
t += h.dist;
```

